

CHAPTER 03

OpenCV 주요 클래스

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

3장 OpenCV 주요 클래스

3.1 기본 자료형 클래스

3.2 Mat 클래스

3.3 Vec과 Scalar 클래스

3.4 InputArray와 OutputArray 클래스

3.3 Vec과 Scalar 클래스

Vec 클래스

- 벡터는 같은 자료형을 가진 다수의 원소로 이루어진 데이터 형식 -> 1차원 배열
- OpenCV는 벡터(1차원배열) 데이터를 저장하는 **Vec** 클래스를 제공함

Vec 클래스

코드 3-14 간략화한 Matx와 Vec 클래스 정의

```

01  template<typename _Tp, int m, int n> class Matx
02  {
03  public:
04      ...
05      _Tp val[m*n]; //< matrix elements
06  };
07
08  template<typename _Tp, int cn> class Vec : public Matx<_Tp, cn, 1>
09  {
10  public:
11      ...
12      /*! element access */
13      _Tp& operator[](int i);
14  };
15
16  template<typename _Tp, int n> static inline
17  std::ostream& operator << (std::ostream& out, const Vec<_Tp, n>& vec);

```

템플릿 매개변수



Vec 클래스

- `Vec` 클래스도 템플릿 클래스 -> 객체를 생성시 `<>` 괄호 안에 벡터 데이터의 자료형(`_Tp`)과 데이터 개수(`cn`)를 명시
- `Vec<int, 4>` : `int`형 데이터 4개를 가지고 있는 벡터를 저장
- `Vec<uchar, 3>` : `uchar`형 데이터 3개를 가지고 있는 벡터를 저장
- 표현을 간단히 하기위해서 `Vec` 템플릿 클래스를 다음 형식으로 재정의하여 사용함

`Vec<num-of-data>{b|s|w|i|f|d}`

- `<num-of-data>` : 벡터 원소의 개수를 의미, 2, 3, 4 등 지정
- `{b|s|w|i|f|d}` : `b` 는 `uchar(unsigned char)`, `s` 는 `short`, `w` 는 `ushort(unsigned short)`, `i`는 `int`, `f`는 `float`, `d`는 `double`형을 의미함

Vec 클래스

- Vec 클래스의 타입 재정의

```
typedef Vec<uchar, 2> Vec2b;  
typedef Vec<uchar, 3> Vec3b;  
typedef Vec<uchar, 4> Vec4b;
```

```
typedef Vec<int, 2> Vec2i;  
typedef Vec<int, 3> Vec3i;  
typedef Vec<int, 4> Vec4i;
```

```
typedef Vec<short, 2> Vec2s;  
typedef Vec<short, 3> Vec3s;  
typedef Vec<short, 4> Vec4s;
```

```
typedef Vec<float, 2> Vec2f;  
typedef Vec<float, 3> Vec3f;  
typedef Vec<float, 4> Vec4f;
```

```
typedef Vec<ushort, 2> Vec2w;  
typedef Vec<ushort, 3> Vec3w;  
typedef Vec<ushort, 4> Vec4w;
```

```
typedef Vec<double, 2> Vec2d;  
typedef Vec<double, 3> Vec3d;  
typedef Vec<double, 4> Vec4d;
```

Vec3b



Vec3b 클래스

- `Vec<uchar, 3>` : uchar형 데이터 3개를 가지고 있는 Vec 클래스
- 컬러 영상의 픽셀 값의 자료형으로 Vec3b 클래스를 사용함
- 컬러 영상의 픽셀 값(B,G,R)을 저장하는 용도로 사용됨
- Vec3b 클래스 객체 생성 및 초기화

`Vec3b p1, p2(0, 0, 255);`

p1	0	0	0
p2	0	0	255

- `[ ]` 연산자함수를 이용하여 멤버 변수에 접근

```
p1[0] = 100;
p1[1] = 150;
p1[2] = 200;
cout << (int)p2[0] << endl;
cout << (int)p2[1] << endl;
cout << (int)p2[2] << endl;
```

p1	100	150	200
	p1[0]	p1[1]	p1[2]

Vec3b 클래스

- std::cout과 << 연산자를 이용하여 Vec 클래스의 원소를 출력

```
cout << "p1: " << p1 << endl;
```

```
cout << "p2: " << p2 << endl;
```

- 실행 결과

```
p1: [100, 0, 0]
```

```
p2: [0, 0, 255]
```

예제 1: Vec3b 클래스

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Vec3b p1, p2(0, 0, 255);
    cout << "p1: " << p1 << endl;
    cout << "p2: " << p2 << endl;
    p1[0] = 100;
    p1[1] = 150;
    p1[2] = 200;
    p2[0] = 100;
    p2[1] = 150;
    p2[2] = 200;
    cout << "p1: " << p1 << endl;
    cout << "p2: " << p2 << endl;
    return 0;
}
```

Microsoft Visual Studio 디버...

```
p1: [0, 0, 0]
p2: [0, 0, 255]
p1: [100, 150, 200]
p2: [100, 150, 200]
```

Scalar 클래스

- `Scalar_` 클래스는 크기 4인 `Vec` 클래스를 상속받은 템플릿 클래스
- `Scalar` 클래스는 크기가 4인 `double`형 배열을 멤버 변수로 가지고 있는 `Scalar_` 클래스를 재정의 한 것

코드 3-15 간략화한 `Scalar_`와 `Scalar` 클래스 정의

```

01  template<typename _Tp> class Scalar_ : public Vec<_Tp, 4>
02  {
03  public:
04      Scalar_();
05      Scalar_(_Tp v0);
06      Scalar_(_Tp v0, _Tp v1, _Tp v2=0, _Tp v3=0);
07
08      static Scalar_<_Tp> all(_Tp v0);
09      ...
10  };
11
12  typedef Scalar_<double> Scalar;

```

템플릿 매개변수

Scalar 클래스

- **Scalar** 클래스는 영상의 픽셀 값을 저장하는 용도로 사용됨
- 그레이스케일 영상의 경우, **Scalar** 클래스의 첫번째 원소가 픽셀 밝기를 표현하고 나머지 3개의 원소는 0으로 설정됨
- 컬러 영상의 경우, **Scalar** 클래스의 처음 3개 원소에 B(파란색), G(녹색), R(빨간색) 색상 성분 값을 저장하고, 4번째 원소는 보통 0으로 설정
- 컬러영상 중 PNG 형식처럼 투명도를 저장하는 경우 **Scalar** 클래스의 네 번째 원소를 이용함
- `std::cout`과 `<<` 연산자를 이용하여 **Scalar**클래스의 원소를 출력할 수 있음

그레이스케일 컬러영상 {	밝기값	0	0	0
	B	G	R	0
	B	G	R	투명도

코드3-16 : Scalar 클래스

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Scalar gray(128);
    cout << "gray: " << gray << endl;

    Scalar yellow(0, 255, 255);
    cout << "yellow: " << yellow << endl;

    yellow[0] = 128;
    yellow[1] = 255;
    yellow[2] = 128;
    yellow[3] = 255;
    cout << "yellow: " << yellow << endl;
    return 0;
}
```

Microsoft Visual Studio 디버그 콘솔

```
gray: [128, 0, 0, 0]
yellow: [0, 255, 255, 0]
yellow: [128, 255, 128, 255]
```

// 밝기값

// B,G,R 순서임

3.4 InputArray와 OutputArray 클래스

InputArray 클래스

- OpenCV 함수의 매개변수의 자료형으로만 사용됨(인터페이스 클래스)
- 사용자가 명시적으로 `InputArray` 객체를 생성하는 것은 금지됨
- `InputArray` 클래스 타입의 객체에는 `Mat`, `Mat_<T>`, `Matx<T, m, n>`, `vector<T>`, `vector<vector<T>>`, `vector<Mat>`, `vector<Mat_<T>>`, `UMat`, `vector<UMat>`, `double` 타입의 객체를 대입할 수 있음
- `InputArray` 타입의 매개변수를 갖는 OpenCV 함수를 호출할 때 `Mat` 또는 `vector<T>` 같은 타입의 객체를 인자로 전달

```
void imshow(const String& winname, InputArray mat);
```

- `InputArray` 클래스는 다음과 같은 형태로 자료형이 재정의되어 있음

```
typedef const _InputArray& InputArray;
```

InputArray 클래스

- 함수 호출시 `InputArray` 타입의 매개변수에 `Mat` 타입의 객체가 전달됨

```
Mat img;
...
imshow("image", img);
...

void imshow(const String& winname, InputArray mat)
{
    ...
    ...
}

mat = InputArray(img);
//생성자 호출하여 Mat객체를
InputArray 임시객체로 변환후 대입(자동형변환과 유사)
```


OutputArray 클래스

- OpenCV 함수의 매개변수의 자료형으로 사용됨
- **OutputArray** 클래스는 많은 OpenCV 함수에서 영상을 입력으로 받아 영상 처리를 수행하고 그 결과를 전달하는 매개변수로 사용됨
- **OutputArray** 타입의 매개변수를 갖는 OpenCV 함수의 호출시 인자로서 Mat 또는 `vector<T>` 같은 타입의 객체를 전달해야 함

```
void cvtColor(InputArray src, OutputArray dst, int code, int dstCn = 0);
```

- **OutputArray** 클래스는 다음과 같이 자료형이 재정의가 되어 있음

```
typedef const _OutputArray& OutputArray;
```

OutputArray 클래스

- 함수 호출시 **InputArray** 타입의 매개변수에 Mat 객체를 전달하고 함수에서 처리한 결과를 **OutputArray** 타입의 매개변수를 통하여 다시 Mat 객체에 저장한다.

Mat matsrc;

Mat matdst;

...

cvtColor(matsrc, matdst, COLOR_BGR2GRAY);

...

```
void cvtColor(InputArray src, OutputArray dst, int code, int dstcn)
{
    ...
    ...
}
```

InputOutputArray 클래스

- `InputOutputArray` 클래스는 입력과 출력의 역할을 동시에 수행하는 OpenCV 함수의 매개변수의 자료형으로 사용됨
- 영상에 그림을 그리는 OpenCV 함수는 입력 영상 자체를 변경하여 다시 출력으로 반환하는 경우가 있으며, 이러한 함수는 `InputOutputArray` 클래스 타입의 매개변수를 사용함
- OpenCV 함수를 호출할 때 `InputOutputArray` 타입의 매개변수에는 `Mat` 또는 `vector<T>` 같은 타입의 객체를 인자로 전달해야 함

```
void line(InputOutputArray img, Point pt1, Point pt2, const Scalar& color,  
          int thickness = 1, int lineType = LINE_8, int shift = 0);
```

InputOutputArray 클래스

- 함수 호출시 `InputOutputArray` 타입의 매개변수에 `Mat` 타입의 객체가 전달됨 -> 입력과 출력 역할을 동시에 함

```
Mat img;
```

```
...
```

```
line(img, Point(50, 50), Point(200, 50), Scalar(0, 0, 255));
```

```
...
```

```
void line(InputOutputArray mat, Point pt1, Point pt2, .... )
{
    ...
    ...
    ...
}
```

실습과제1

- Vec3b 클래스를 이용하여 키보드로부터 B,G,R 컬러값을 입력 받아 저장하고 화면에 출력하는 프로그램을 작성하시오.

Blue값(0~255)을 입력하시오: 255 <엔터>
Green값(0~255)을 입력하시오: 0 <엔터>
Red값(0~255)을 입력하시오: 255 <엔터>
입력한 컬러값은 [255, 0, 255] 입니다.

실습과제2

- Scalar 클래스를 이용하여 키보드로부터 B,G,R 컬러값을 입력 받아 저장하고 화면에 출력하는 프로그램을 작성하시오.

Blue값(0~255)을 입력하시오: 255 <엔터>
Green값(0~255)을 입력하시오: 0 <엔터>
Red값(0~255)을 입력하시오: 255 <엔터>
입력한 컬러값은 [255, 0, 255, 0] 입니다.

과제 제출 방법

- 소스파일과 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.